16-662 Robot Autonomy: Top 3 Project Ideas -- Full Comparison

Context

Course: 16-662 Robot Autonomy, Prof. Oliver Kroemer (CMU) **Team**: 5 students, Labs 1-2+ completed (comfortable with Franka) **Domain**: Stackable blocks on Franka 7-DOF, two flat areas + shelf, predefined grid slots, NO camera (positions given) **Deliverables**: Presentation (10%), Report 2-3 pages (5%), Video (5%) = 20% total grade **Goal**: Identify the project that maximizes demonstrated mastery across all 14 lecture topics

---

What Kroemer Implicitly Rewards (Decoded from Slides)

1. **Full autonomy hierarchy** (slides 16-22): Task Context -> Symbolic Planner -> Skill (Init/Plan/Term) -> Controller -> Robot -> Feedback -> Learner. A project that visibly implements ALL layers will stand out.
2. **Robustness** (slide 3): "Recover without assistance if possible" -- error recovery is not optional for top marks.
3. **Modularity** (slide 23): Reusable components, clean abstraction layers.
4. **Learning** (his research): He develops learning algorithms for manipulation. Projects with a learning component speak his language.
5. **Honest limitations** (slide 44): "Be clear on limitations and assumptions." He respects teams that know what they don't know.
6. **Ambition with execution** (slide 48): "Tailor project to interests and risks." He wants you to swing -- but connect.

Lecture Topics Scorecard (14 total)

#	Topic	Abbrev
1	Forward/Inverse Kinematics	FK/IK
2	Control (PID, impedance)	CTRL
3	C-Space + Collision Detection	COLL
4	Probabilistic Roadmaps	PRM
5	Rapidly-exploring Random	RRT

#	Topic	Abbrev
	Trees	
6	Planning with Constraints	CNST
7	Discrete Search (A*, BFS, etc.)	DSRCH
8	Hybrid Systems	HYB
9	Task Planning (STRIPS, symbolic)	TASK
10	ML / NNs / GPs	ML
11	MDPs	MDP
12	Imitation Learning	IL
13	Reinforcement Learning	RL
14	Interactive Perception / Filtering	PERC

IDEA 1: Autonomous Chess Board Reset

Elevator Pitch

Given a post-game chess board (pieces scattered across the 8x8 grid), the robot autonomously computes the optimal move sequence and returns every piece to its starting position -- handling dependencies (piece A's target is occupied by piece B), planning collision-free paths through the cluttered board, and recovering from grasp failures.

Why This Idea

- **Proven feasible**: Slide 39 shows a chess board project from a past year
- **Highest topic coverage**: 12/14 lecture topics naturally arise
- **Clean narrative**: Everyone understands chess -- presentation and video sell themselves
- **Natural complexity graduation**: Start with 4 pieces, scale to 16, then 32
- **The combinatorial planning problem is intellectually non-trivial**: move ordering with dependencies is essentially a constraint-satisfaction + graph search problem that directly

showcases discrete search AND task planning

Topic Coverage: 12/14

Topic	How It's Used	Depth
FK/IK	Compute joint angles for 64 board positions + staging area	Core
CTRL	Impedance control for gentle placement; force-controlled grasps for pieces of varying heights (pawn=short block, king=tall block)	Core
COLL	Other pieces on the board ARE obstacles; collision checking between gripper+held piece and standing pieces	Core
PRM	Build roadmap of safe transit configurations above the board; reuse across multiple moves	Core
RRT	RRT-Connect for individual move planning when PRM graph is stale (after board state changes)	Core
CNST	Orientation constraints (pieces must remain upright), gripper clearance near board edges, approach-angle constraints for pieces next to the shelf wall	Core
DSRCH	**The star of the show**: Computing optimal move ordering. If piece A needs to go to square (3,4) but piece B currently sits there, B must move first. Build a dependency graph; find a valid topological ordering; optimize for minimum total robot travel distance (variant of TSP on the valid orderings)	Core
HYB	Discrete FSM (idle -> select_piece -> plan_path -> pick -> transit -> place -> verify -> next) interleaved with continuous trajectory execution	Core
TASK	STRIPS-style symbolic planning: `preconditions: {square_empty(target), piece_graspable(p)}`, `effects: {piece_at(p, target), square_empty(source)}`	Core
MDP	Model placement success probability as function of board clutter; choose between "direct placement" vs. "clear neighbors first"	Supporting
IL	Learn grasp height/approach parameters for each piece type from 3-5 kinesthetic demos per type	Supporting
PERC	Torque-based grasp success detection; contact detection during placement	Supporting
~~ML~~	Not directly needed (covered via IL)	--
~~RL~~	Could add as stretch (refine placement precision via trial-and-error)	Stretch

Architecture (Maps Exactly to Kroemer's Hierarchy Diagram)

```
[Task Context]

  "Reset board: current_state -> initial_state"

  Input: 64-element state array (piece_type per cell)

        |

[Symbolic Planner / Task Planner]  ---- DSRCH + TASK

  1. Build dependency graph: for each piece, check if its target square is occupied

  2. Topological sort to find valid move orderings

  3. Optimize ordering for min total distance (greedy nearest-neighbor heuristic on valid
orderings)

  Output: ordered list [(piece_id, source, target), ...]

        |

[Selected Skill: PICK_AND_PLACE]  ---- HYB

  - Initialization: compute approach pose for source square (FK/IK)

    # FK/IK

  - Motion Planner: plan collision-free path from current config

    # PRM/RRT + COLL

    to approach pose, through board obstacle field

  - Grasp: impedance-controlled descent, torque-threshold close

    # CTRL + PERC

  - Transit: plan path from source to target (holding piece)

    # RRT + CNST

    with HELD PIECE as additional collision geometry

  - Place: impedance-controlled descent to target square

    # CTRL

  - Termination: torque check for successful placement

    # PERC

  - Error handler: if grasp fails -> retry with adjusted params

    # MDP (risk assessment)

                    if drop during transit -> replan from current state

                      # TASK (replan)
```

```
        |
[Skill Controller]  ---- CTRL
  Joint-level impedance control, trajectory tracking
        |
[Franka Robot] <-> [Board Environment]
        |
[Feedback]  ---- PERC
  Joint torque readings -> grasp success, contact detection, collision detection
        |
[Learner]  ---- IL
  Log (piece_type, grasp_params, success) -> adapt grasp height/force per piece type
```

Work Breakdown (5 people, ~6 weeks)

Member	Responsibility	Weeks
P1: Board & State Rep	Define board coordinate frame, map 64 squares to (x,y,z) in robot frame, implement state representation, build block-to-piece mapping	W1-2
P2: Discrete Planner	Dependency graph construction, topological sort, distance-optimized ordering, STRIPS formulation	W1-3
P3: Motion Planner	PRM/RRT integration with dynamic obstacle field (pieces on board), collision checking with held piece geometry	W2-4
P4: Skill Controller	Impedance-controlled pick/place primitives, torque-based success detection, error recovery FSM	W2-4
P5: Integration & Learning	Full pipeline integration, IL module for grasp adaptation, testing & debugging, video production	W3-6

All: Final presentation, report writing (W5-6)

Risk Assessment

Risk	Likelihood	Mitigation
Pieces too small to grasp reliably	Medium	Use blocks of adequate size (2-3cm); tune impedance params
Move ordering solver too slow for 32 pieces	Low	Greedy heuristic is O(n^2); exact only needed for small boards

Risk	Likelihood	Mitigation
Collision with adjacent pieces during transit	Medium	Conservative clearance heights; approach from above
Shared robot scheduling conflicts	High	Develop in simulation first (PyBullet/MuJoCo with Franka URDF); minimize real-robot time

Demo Scenario (for video)

1. Show the "messy" board (post-game state with pieces scattered)
2. Robot computes move plan (visualize dependency graph on screen)
3. Robot executes: picks each piece, navigates around obstacles, places precisely
4. Handle a deliberate "failure" (manually shift a piece mid-execution) -> robot detects and replans
5. Board is fully reset. Camera pans to show clean starting position.

IDEA 2: Adaptive Block Sorting via Interactive Perception

Elevator Pitch

Blocks of identical appearance but different hidden properties (weight, friction coefficient, stiffness) arrive at a staging area. The robot must DISCOVER each block's properties through physical interaction -- lifting to measure weight (torque sensing), pushing to measure friction, pressing to measure compliance -- then sort them into the correct shelf locations. The robot actively decides WHICH test to run next to maximize information gain (active perception via MDP).

Why This Idea

- **Directly aligned with Kroemer's research** (interactive perception + learning for manipulation)
- **Uniquely exploits the "no camera" constraint**: turns a limitation into the project's thesis
- **Uses the Franka's torque sensing** -- a capability he specifically highlighted (slide 38)
- **Intellectually sophisticated**: active information gathering under uncertainty is a deep topic
- **Novel among student projects**: past projects (slide 40) are mostly pick-and-place; nobody has done perception-driven sorting

Topic Coverage: 12/14

Topic	How It's Used	Depth
FK/IK	Reaching staging area, shelf locations, test poses	Core
CTRL	Force-controlled probing: lift with measured force, push	Core

Topic	How It's Used	Depth
	with measured resistance, press with measured compliance	
COLL	Shelf collision avoidance during placement, table edge awareness during push tests	Core
PRM	Path planning between staging area, test locations, and shelf	Supporting
RRT	Alternative planner for constrained shelf placements	Supporting
CNST	Force limits during probing (don't damage blocks), shelf placement constraints (heavy on bottom shelf, light on top)	Core
DSRCH	Optimal test sequence selection when tests have dependencies (must lift before you can push while holding)	Supporting
HYB	State machine: (identify -> test -> classify -> sort) with loops back when uncertain	Core
TASK	Symbolic: `test(block, weight) -> if heavy: place(block, bottom_shelf)`	Core
ML/GP	**Gaussian Processes for property estimation**: model P(weight	lift_torque) with uncertainty bounds. After each test, update posterior. Classify when uncertainty drops below threshold
MDP	**Active perception as MDP**: state = current belief about block properties, actions = {lift, push, press, classify}, reward = correct classification, cost = time per test. Solve for optimal test policy	Core
IL	Learn probing motions from kinesthetic demos (how to lift/push/press safely)	Supporting
RL	Learn the active perception policy: which test to run next	Core
PERC	**The entire project IS interactive perception** -- using robot's own actions to gather information about the world	Core

Architecture

```
[Task Context]
```

```
  "Sort N blocks from staging area to categorized shelf locations"

  Input: N blocks at known positions, unknown properties

        |

[Active Perception Planner (MDP/RL)]  ---- MDP + RL

  For each unclassified block:

    1. Current belief state: P(category | tests_so_far) via GP

    2. Select next action: {lift_test, push_test, press_test, classify_and_sort}

    3. If classify: select shelf location based on estimated category

        |

[Selected Skill: {LIFT_TEST, PUSH_TEST, PRESS_TEST, PICK_AND_SORT}]  ---- HYB

  LIFT_TEST:

    - Approach block, grasp, lift to height h

      # FK/IK + CTRL

    - Record joint torques during lift -> estimate weight          ---- PERC

    - Place block back down                                        ---- CTRL

    - Update GP posterior: P(weight | torque_measurements)         ---- ML/GP

  PUSH_TEST:

    - Position gripper at block edge                               ---- FK/IK

    - Push block distance d along surface                          ---- CTRL

    - Record force profile -> estimate friction coefficient        ---- PERC

    - Update GP posterior: P(friction | force_profile)             ---- ML/GP

  PRESS_TEST:

    - Position gripper above block                                 ---- FK/IK

    - Press down with increasing force, record deflection

      # CTRL + PERC

    - Estimate stiffness from force-deflection curve               ---- ML/GP

  PICK_AND_SORT:

    - Grasp block, plan path to target shelf slot

      # PRM/RRT + COLL + CNST

    - Place on shelf                                               ---- CTRL
```

```
        |
[Skill Controller]  ---- CTRL
  Impedance control with force measurement during probing
        |
[Franka Robot] <-> [Blocks + Shelf]
        |
[Feedback]  ---- PERC
  Joint torques, end-effector forces, position tracking
        |
[Learner]  ---- GP + RL
  GP: update property estimates after each test
  RL: improve test selection policy over episodes
```

Work Breakdown (5 people, ~6 weeks)

Member	Responsibility	Weeks
P1: Probing Skills	Implement lift/push/press primitives with force recording, kinesthetic demo collection	W1-3
P2: Gaussian Processes	GP model for property estimation, posterior updates, uncertainty quantification, classification thresholds	W1-3
P3: Active Perception MDP	Formulate MDP, implement value iteration / Q-learning for test selection policy	W2-4
P4: Motion Planning + Sorting	Shelf placement planner, category-to-location mapping, collision-free paths	W2-4
P5: Integration + Evaluation	End-to-end pipeline, ablation studies (random test order vs. learned policy), video production	W3-6

Risk Assessment

Risk	Likelihood	Mitigation
Torque sensing too noisy for reliable weight estimation	Medium	Average over multiple measurements; use GP uncertainty to trigger re-tests
Friction test causes block to slide off table	Medium	Constrain push distance; add table-edge safety checks

Risk	Likelihood	Mitigation
GP needs too much training data	Low	Use prior from physics (expected weight ranges); few-shot GP works well
RL policy doesn't converge in limited robot time	Medium	Train in simulation first; real robot for validation only

Demo Scenario (for video)

1. Show 8 identical-looking blocks on staging area. "These look the same, but they're all different."
2. Robot picks up block 1, lifts it. Overlay: "Estimated weight: 150g +/- 20g (uncertain)." Robot pushes it. Overlay: "Estimated weight: 155g +/- 5g. Friction: high. Classification: HEAVY-ROUGH -> Bottom shelf."
3. Fast-forward through remaining blocks, showing the GP uncertainty shrinking with each test.
4. Show how the learned policy skips unnecessary tests (e.g., "This block is clearly light from the lift test alone -- no need to push, classify immediately").
5. End: shelf neatly organized by category. Overlay: "12 tests total vs. 24 if testing everything -- 50% more efficient with learned policy."

IDEA 3: Autonomous Block Structure Builder with Symbolic Planning & Error Recovery

Elevator Pitch

The robot receives a high-level specification of a block structure (e.g., "3-layer pyramid on the left platform, L-shape on the shelf") and autonomously plans the build order (bottom-up, back-to-front), executes collision-free manipulation, and detects + recovers from construction errors (toppled blocks, misplacements) using torque feedback. The planning layer uses STRIPS-style task planning; the execution layer chains motion primitives; the recovery layer replans from the observed failure state.

Why This Idea

- **Purest expression of the hierarchical autonomy stack** -- every layer of Kroemer's diagram is exercised
- **Build order planning is a rich combinatorial problem**: stability constraints (can't place top before bottom), reachability constraints (back row before front row on shelf), and resource constraints (one gripper)
- **Error recovery is the differentiator**: most student projects work in happy-path mode; this one is DESIGNED to handle failures
- **Very strong video**: watching a robot build structures, detect a failure, and autonomously fix it is

visually compelling

- **Scalable complexity**: start with 3-block stack, end with 10+ block composite structure across multiple surfaces

Topic Coverage: 11/14

Topic	How It's Used	Depth
FK/IK	Reaching source blocks, computing placement poses at varying heights (stacking = progressively higher targets)	Core
CTRL	Impedance control critical for stacking: gentle placement to avoid toppling lower blocks; force feedback for seating confirmation	Core
COLL	Partially-built structures are obstacles; collision checking for approach/retract around existing tower	Core
PRM	Roadmap for navigating between source area, left platform, shelf	Supporting
RRT	Path planning around partially-built structures (dynamic obstacles)	Core
CNST	**Stability constraints**: can't place block if support below isn't stable; **reachability**: shelf back-row must be placed before front-row blocks occlude it; **orientation**: blocks must be level	Core
DSRCH	Build order optimization: find ordering that satisfies all constraints AND minimizes total robot travel + risk of toppling	Core
HYB	Multi-phase FSM: (plan_build_order -> pick_next_block -> plan_path -> pick -> transit -> place -> verify_stability -> if_fail: replan)	Core
TASK	STRIPS planning: `action: stack(B, on_top_of_A)`, `precond: {holding(B), clear(A), stable(A)}`, `effect: {on(B,A), clear(B), not holding(B)}`. Goal: target structure described symbolically	Core
MDP	Model probability of toppling as function of tower height and placement precision; decide whether to attempt direct stack vs. reinforce base first	Supporting
IL	Learn stacking primitives (approach angle, descent speed, release timing) from kinesthetic demos	Supporting
~~RL~~	Not primary (could be stretch goal for refining stacking precision)	Stretch
PERC	Torque-based verification: "did the block seat properly?" -- detect if placed block is sitting level or tilted (asymmetric torque profile = tilt)	Core
~~ML/GP~~	Not directly needed	--

Architecture

```
[Task Context]

  "Build structure S on platform P using blocks from source area"

  Input: structure specification as symbolic goal state

         e.g., {on(B2, B1), on(B3, B2), on_platform(B1, left)}  -- 3-block tower

        |

[STRIPS Task Planner]  ---- TASK + DSRCH

  1. Parse goal into required actions: [place(B1, left_platform), stack(B2, B1), stack(B3, B2)]

  2. Compute valid orderings via dependency analysis

  3. Add reachability constraints (shelf: back-to-front; platform: bottom-to-top)

  4. Output: ordered action sequence

        |

[Selected Skill: {PICK_BLOCK, PLACE_ON_SURFACE, STACK_ON_BLOCK}]  ---- HYB

  PICK_BLOCK:

    - Compute approach pose for source block (FK/IK)

    - Plan collision-free path to source (RRT, avoiding built structures)

      # RRT + COLL

    - Impedance-controlled grasp with torque confirmation

      # CTRL + PERC

  PLACE_ON_SURFACE:

    - Plan path to target location (PRM/RRT + CNST)

    - Impedance-controlled descent with force limit

    - Release + retract + verify placement via torque signature

  STACK_ON_BLOCK:

    - CRITICAL SKILL: must be very gentle to avoid toppling tower

    - Plan approach: come from directly above, slow descent

      # CNST

    - Impedance-controlled placement with reduced stiffness

      # CTRL

    - Force-based seating detection: "did it click into place?"
```

```
      # PERC
    - If asymmetric torque detected -> block is tilted -> trigger replan
        |
[Error Recovery Module]  ---- HYB + TASK
  Monitors for failures:
    1. Grasp failure (empty gripper torque) -> retry with adjusted params
    2. Stack failure (tilt detected) -> remove tilted block, re-attempt
    3. Tower collapse (sudden torque spike during placement) ->
       re-assess current state (which blocks are still placed correctly?)
       -> replan from current state to goal state
        |
[Skill Controller]  ---- CTRL
  Variable-impedance control: stiff for picks, compliant for stacks
        |
[Franka Robot] <-> [Blocks + Platforms + Shelf]
        |
[Feedback]  ---- PERC
  Joint torques for grasp/place/stack success detection
        |
[Learner]  ---- IL
  Stacking demos: learn descent speed, release timing, impedance params per tower height
```

Work Breakdown (5 people, ~6 weeks)

Member	Responsibility	Weeks
P1: Task Planner	STRIPS formulation, dependency graph, build order optimizer with stability + reachability constraints	W1-3
P2: Motion Planning	RRT-Connect with dynamic obstacle field (growing structure), approach-from-above constraint for stacking	W1-3
P3: Manipulation Skills	Pick, place, stack primitives with impedance control tuning; kinesthetic demo collection for stacking	W2-4

Member	Responsibility	Weeks
P4: Error Detection & Recovery	Torque-based failure detection, state re-estimation after failure, replanning integration	W2-5
P5: Integration & Structures	Define 5+ target structures of increasing complexity, end-to-end testing, video production	W3-6

Risk Assessment

Risk	Likelihood	Mitigation
Stacking 3+ blocks is unreliable	Medium-High	Tune impedance params aggressively; use blocks with flat/stable surfaces; limit to 3-4 layers
Collapse detection is imprecise	Medium	Combine torque monitoring with expected placement time -- if placement takes too long, assume failure
Task planner gets complex with many blocks	Low	Limit to 8-10 blocks; STRIPS solver handles this scale easily
Error recovery causes infinite retry loops	Low	Max retry count per action (3); escalate to "skip and continue" after max retries

Demo Scenario (for video)

1. "Task: Build a pyramid on the left platform and an L-shape on the shelf."
2. Robot plans: show the task plan + build order on screen
3. Robot picks blocks from source, builds pyramid bottom-up. Smooth stacking.
4. Halfway through shelf construction, demonstrator INTENTIONALLY nudges a block (simulating error)
5. Robot detects the shift via torque feedback. Pauses. Replans.
6. Robot corrects: removes tilted block, replaces it, continues building.
7. Final shot: both structures complete. "Planned 12 moves. Executed 15 (3 recovery actions)."

Head-to-Head Comparison

Dimension	Chess Board Reset	Interactive Perception	Structure Builder
Topics Hit	12/14	12/14	11/14
Core Innovation	Combinatorial move planning with dependencies	Active property discovery under uncertainty	Error recovery & replanning
Kroemer	High (manipulation + planning)	**Very High** (his exact	High (hierarchical

Dimension	Chess Board Reset	Interactive Perception	Structure Builder
Research Fit		research area)	autonomy)
Feasibility	**High** (proven by past team)	Medium-High (torque sensing noise is a risk)	Medium (stacking reliability)
Visual Impact (Video)	**Very High** (everyone knows chess)	High (uncertainty shrinking is cool)	**Very High** (building + recovery)
Novelty	Medium (done before, but can improve)	**Very High** (no past team did this)	High (error recovery focus is rare)
Risk	Low-Medium	Medium	Medium-High
5-Person Parallelism	Excellent (planner/motion/control/learning/integration cleanly separate)	Excellent (probing/GP/MDP/sorting/integration)	Good (planner/motion/skills/recovery/integration)
"Wow" Ceiling	Board with 32 pieces reset flawlessly	Robot figures out hidden properties with minimal tests	Robot builds complex structure and self-corrects

My Ranking

1. Chess Board Reset -- Best overall balance. Lowest risk, highest topic coverage, proven feasible, cleanest narrative. The combinatorial move-ordering problem is genuinely impressive to present. A past team did it, so you know it works -- your job is to add the learning + error recovery layer they likely didn't have, and plan optimally rather than brute-force.

2. Interactive Perception Sorting -- Highest ceiling and most aligned with Kroemer's personal research. If executed well, this is the project that makes him think "this team really understood what I care about." But the torque-sensing reliability risk is real. If your team is comfortable with uncertainty modeling (GPs, MDPs), this could leapfrog #1.

3. Structure Builder -- Purest expression of hierarchical autonomy with the best error-recovery narrative. But stacking reliability is a known pain point with real robots, and if stacking doesn't work reliably, the whole demo suffers. Best for a team that wants to wrestle with real-robot messiness.

The Hybrid Play (Secret A+ Move)

If you want to truly overdeliver: **pick Chess Board Reset as the base**, but add one Interactive Perception element. For example: pieces (blocks) of different weights need to be identified before the robot knows which starting square they belong to. The robot lifts each piece, estimates weight via torque sensing, classifies it (pawn=light, king=heavy), and THEN plans the reset sequence. This adds PERC + GP + MDP to an already 12-topic project, pushing to **14/14 topic coverage**.